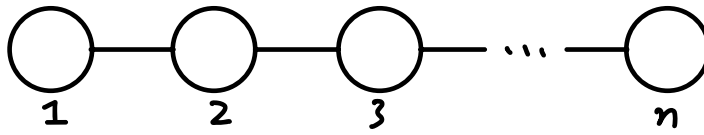


Exercice 1. Diffusion dans une chaîne

On considère une chaîne de n sites. On distingue la racine et la feuille de cette chaîne. De plus, on suppose que chaque site i connaît l'ensemble de ses voisins, noté $vois_i$.

1. Écrire un algorithme distribué permettant la diffusion d'un message de la racine à la feuille. Indiquer dans l'algorithme l'endroit où la terminaison est détectée.
2. Donner la complexité en messages de cet algorithme.
3. Donner la complexité en terme de rondes d'exécutions (dans le cas synchrone).

Algo Chaîne :



variables connaissances :

[
Vois_i : les noeuds voisins du noeud i .
racine_i : vraie pour la racine
faux sinon.
feuille_i : vrai pour les feuilles
faux sinon
]

variable locales

[info_i : le truc à transmettre

variables des communication

[

Code :

Événement initial : sur 1 noeud de racine_i = vraie.

[Aller(max_i) à {Vois_i}

Réception Aller(m) (variables de message) de j (noeud voisin).

[
Si $m > \max_i$
[$\max_i \leftarrow m$
Si $|Vois_i| > 1$
[Envoyer Aller à {vois_i} \ { j }
sinon
Envoyer Retour(max_i) à j
]

Réception de Retour(v) de j

[
 $\max_i \leftarrow v$
Si $|Vois_i| > 1$
[Envoyer Retour(max_i) à vois_i \ j
]

Exercice 3. Remontée d'information dans un arbre

On considère un arbre de n sites. Le but de l'exercice est de concevoir un algorithme distribué permettant la remontée d'une information des feuilles vers la racine (algorithme inverse de la diffusion). Écrire une version de l'algorithme pour les cas suivants :

1. Chaque site i sait distinguer son père ($père_i$) et ses fils (ensemble $fils_i$) dans l'arbre. Indiquer dans l'algorithme l'endroit où la terminaison est détectée.
2. Chaque site ne connaît que l'ensemble de ses voisins dans l'arbre ($vois_i$). Un site ne sait donc pas faire la distinction entre son père et ses fils. La racine de l'arbre est distinguée. Votre algorithme doit calculer explicitement le père et les fils de tous les sites internes de l'arbre. Indiquer également l'endroit où la terminaison est détectée.
3. Comme dans le cas précédent, chaque site ne connaît que l'ensemble de ses voisins dans l'arbre ($vois_i$). En revanche, ici, la racine de l'arbre n'est pas distinguée. Votre algorithme doit calculer explicitement le père et les fils de tous les sites internes de l'arbre. Pourquoi ne peut-on pas indiquer dans l'algorithme l'endroit où la terminaison est détectée ?

1)
Connaissance :

$père_i$: père de nœud i dans l'arbre

$fils_i$: fils de nœud i dans l'arbre

(Implicitement : Racine : $père_i = \text{null}$; feuille e : $fils_e = \text{null}$)

$info_i$: l'information à transmettre

 null pour tous les nœuds sauf feuilles

variable :

$valeur_i$: stocker l'information reçue des feuilles

nb_fils

Message : $Msg(info)$

Algorithme 1 :

Initialement: "sur les feuilles"

Si $fils_i = \text{null}$ # sur tous les nœuds

 | Envoyer $Msg(info_i)$ à père i

 | stop local

Sur Réception de $Msg(info_j)$ de j

nb_fils++

$valeur_i \leftarrow info_j$

Si $père_i \neq \emptyset$ et $nb_fils = |fils_i|$

 | Envoyer $Msg(info)$ à père i

 | stop-local

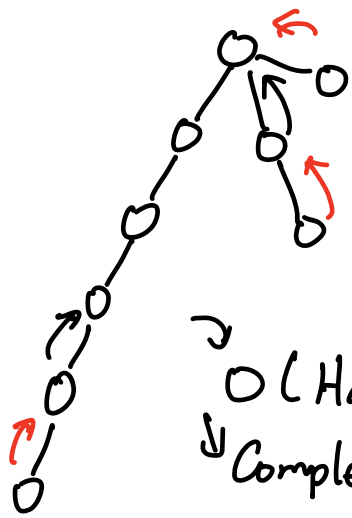
Si $père_i = \text{null}$ et $nb_fils = |fils_i|$

 | stop global

Complexité de message : 1 message par canal . la communication

→ 1 arbre avec n sites $\Rightarrow n-1$ canaux.

En synchrone, complexité en temps



C^0 : config initiale

C^1 : 1ere configuration

↓ (Hauteur de l'arbre)

↓ Complexité de temps synchrone = # configuration.

2)

Connaissance :

$Racine_i$: null sur tous les nœuds sauf 1 .

$Vois_i$: (simplement : feuille : $|Vois_i| = 1$ et $Racine_i = null$)

info : l'information à transmettre

null pour tous les nœuds sauf feuilles.

Variables :

$valeur_i$: stocker l'information reçu des feuilles.

$père_i$: } il faut les construire.

$fils_i$: }

Variables de communication : $Msg(info)$

Algo 2

Initialement : "sur les feuilles"

Si $info \neq null$ alors

Envoyer $Msg(info_i)$ à $Vois_i$.

stop-local

Sur réception de $Msg(info)$ de j

$fils_i \neq j$ }

Si $|Vois_i| - |fils_i| = 1$ et $racine_i = \text{null}$

$père_i \leftarrow \{Vois_i\} \setminus \{fils_i\}$

 Envoyer $Msg(\text{info})$ à $père_i$

 Stop-local

Si $|Vois_i| = |fils_i|$

 Stop-global

3)

Connaissance:

$Vois_i$: (simplement : feuille : $|Vois_i| = 1$)

Variables de communication : $Msg(\text{info})$

info : l'information à transmettre

 null pour tous les nœuds sauf feuilles.

Variables :

$valeur_i$: stocker l'information reçu des feuilles.

$père_i$: il faut les construire.

Algo 3 :

Initialement : " Sur les feuilles "

 Envoyer $Msg(\text{info})$

$père_i \leftarrow Vois_i$

Sur réception de $Msg(v)$ de i :

 Si $père_i = \emptyset$

$fils_i \leftarrow \{j\}$

$valeur_i \leftarrow \text{info}$

 Si $|Vois_i| = |fils_i| + 1$

$père_i \leftarrow Vois_i \setminus fils_i$

 Envoyer $Msg(valeur_i)$ à $père_i$

 Si $père \neq \emptyset$

 Si $i < j$

$père_i \leftarrow \emptyset$

$valeur_i \leftarrow \text{info}$

 Stop-global

Exercice 4. Diffusion avec retour dans un arbre

Écrire un algorithme de diffusion d'information avec retour sur un arbre dont la racine est distinguée. Le déroulement global de l'algorithme est le suivant : la racine diffuse un message à tous les sites de l'arbre. Puis chaque feuille, à la réception de ce message, renvoie un accusé de réception vers son père. L'algorithme est terminé lorsque la racine a reçu un accusé de réception de chacun de ses fils.

On suppose que chaque site i ne connaît que l'ensemble de ces voisins ($vois_i$). Votre algorithme doit calculer explicitement le père et les fils de tous les sites internes de l'arbre. Indiquer également l'endroit où la terminaison est détectée. Enfin, donner la complexité en messages de l'algorithme.

Connaissance :

$Vois_i$: [simplement : feuille : $|Vois_i| = 1$, racine : $= 1$ pour racine)

Variables de communication : $Aller(info)$, $Ret(info)$.

$info$: l'information à transmettre

null pour tous les nœuds sauf feuilles.

Variables :

$père_i$: il faut les construire.

Algo:

Initialement " Sur la racine "

Si $racine_i = 1$

| Envoyer $Aller(info)$ à $Vois_i$

Sur réception de $Aller(info)$ de j

$père_i \leftarrow \{j\}$

Si $|Vois_i| > 1$

| Envoyer $Aller(info)$ à $Vois_i \setminus \{j\}$

Sinon

| Envoyer $Ret(info)$ à j .

Sur réception de $Ret(info)$ de j

$nb_fils ++$

Si $nb_fils = |Vois_i| - 1$ et $racine = null$

| Envoyer $Ret(info)$ à père

| Stop-local

Si $nb_fils = |Vois_i|$ et $racine = 1$

| Stop-global.